

# Building the ShopNest Ticket Intelligence Workflow

A build-along guide — every node, from an empty canvas, and why each one is there.

Written for the Great Learning *Generative and Agentic AI Foundations* project, no-code (n8n) track.

## How to use this

---

This is the workflow you already have, taken apart. Open a **new, empty** workflow next to this document and build it a second time from scratch. You will not break anything — your working copy stays exactly where it is.

Building it yourself once is the difference between having the assignment and understanding it. Each section says what to click, then explains why it is there. The green boxes are the reasoning. The red boxes are the things that actually went wrong while we built it.

## The idea in one paragraph

---

A support ticket arrives as messy free text. We do four things to it: **summarise** it, **score that summary**, **write a reply**, and **score that reply**. Then we write all four results plus the original ticket onto one row of a spreadsheet. The two scoring steps are what make this more than a demo — they turn "the replies look fine" into a number you can track week by week.

```
Run Pipeline → Read Ticket CSV → Extract Ticket Data
                                   |
                                   ▼
Summarization → Evaluation for → Response → Evaluation for
                Summarization   Generation  Response Generation
                                   |
                                   ▼
Extract the Outputs → Convert to File → Save the Data

Each of the four AI boxes carries two smaller boxes underneath:
  an OpenAI Chat Model (which model, how creative, how long)
  a Structured Output Parser (what shape the answer must be)
```

That is 9 main nodes and 9 sub-nodes: **18 in total**.

**Why this shape?** It is not a free choice. Your Solution Approach & Guidelines document specifies these nodes by name, and the marking scheme checks screenshots against them. Where the guidelines name a node, we use their name exactly.

## Before you start

---

- The n8n lab open, and `support_ticket_data.csv` uploaded via the **Files** tab.

- Your lab file path. Ours was `/data/learner-16426312/files/` — yours is on the Files panel, bottom left.
- The credential **Great Learning AI (OpenAI + Gemini)**, which the lab provides.

## Part 1 — The plumbing (3 nodes)

### Node 1 · Run Pipeline

**Add node** → search **Manual Trigger**. Rename it to `Run Pipeline` (double-click the name at the top of the node).

Every workflow needs something that starts it. A manual trigger means "starts when I press Execute Workflow". In production you would swap this for a schedule or a webhook, and nothing downstream would change.

### Node 2 · Read Ticket CSV

**Add node** → **Read/Write Files from Disk**.

| Field            | Value                                                                    |
|------------------|--------------------------------------------------------------------------|
| Operation        | Read File(s) From Disk                                                   |
| File(s) Selector | <code>/data/learner-&lt;your-id&gt;/files/support_ticket_data.csv</code> |

This reads the file off the disk as raw bytes. It does **not** understand that the bytes are a spreadsheet — that is the next node's job.

**Trap 1 — the run that succeeded and did nothing.** On our first attempt this node still held a placeholder path. It found no file, returned **zero items**, and did not raise an error. With no items to work on, n8n skipped all 16 remaining nodes and marked the whole run **successful**. Two lines in the log, nine milliseconds, no output.

The lesson: in n8n, "Success" on a node means only *this node did not throw an error*. It does not mean it did anything. Always check the item count and the duration — 30 tickets through 4 AI stages takes minutes, not milliseconds.

### Node 3 · Extract Ticket Data

**Add node** → **Extract from File**.

| Field                                   | Value             |
|-----------------------------------------|-------------------|
| Operation                               | Extract From CSV  |
| Input Binary Field                      | <code>data</code> |
| Options → Exclude Byte Order Mark (BOM) | <b>ON</b>         |
| Options → Header Row                    | ON                |

This turns the bytes into 30 separate items, one per ticket, each with `support_ticket_id` and `support_ticket_desc`.

**Trap 2 — the invisible character.** This CSV begins with a UTF-8 *byte order mark*: an invisible marker some programs put at the start of a text file. Left alone, it becomes part of the **name** of the first column — so the column is really called `[BOM]support_ticket_id` rather than `support_ticket_id`, where `[BOM]` stands for a character you cannot see on screen at all. Every reference to `support_ticket_id` then returns empty, and **nothing anywhere reports an error**. That is what the Exclude BOM toggle prevents.

**Why one item per ticket matters.** From here on, n8n runs every downstream node once per item, automatically. There is no loop node in this workflow and none is needed — that is n8n's core idea, and it is why the guidelines do not ask for one.

## Part 2 — One AI stage, in full detail

All four AI stages are built the same way. Learn this one and the other three are variations.

### Node 4 · Summarization (the chain)

**Add node → Basic LLM Chain.** Rename to `Summarization`.

| Field                                | Value                                                                                                                                                                   |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Source for Prompt                    | Define below                                                                                                                                                            |
| Prompt (User Message)                | <code>Ticket: "{{ \$('Extract Ticket Data').item.json.support_ticket_desc }}"</code><br>Summary:<br>Return raw JSON only. Do not use markdown code fences or backticks. |
| Require Specific Output Format       | <b>ON</b>                                                                                                                                                               |
| Chat Messages → Add → System Message | the summarisation instructions                                                                                                                                          |

**System message vs prompt.** The **system message** is the job description — who the model is and what rules it follows. It is the same for all 30 tickets. The **prompt** is the individual piece of work — this ticket. Putting the rules in the system message and the data in the prompt is what keeps behaviour consistent across items.

The system message for this stage tells the model: you are a triage analyst; never invent detail; copy identifiers exactly; strip the emotion but keep the facts; expand shorthand; 2-4 sentences.

### Node 5 · Summarisation Model (sub-node)

Underneath the chain node there is a connector marked **Model**. Click the **+** on it and add an **OpenAI Chat Model**.

| Field | Value |
|-------|-------|
|-------|-------|

|                          |                                     |
|--------------------------|-------------------------------------|
| Credential               | Great Learning AI (OpenAI + Gemini) |
| Model                    | gpt-4o-mini                         |
| Sampling Temperature     | 0.2                                 |
| Maximum Number of Tokens | 400                                 |

**What temperature actually does.** It controls how much the model varies its wording. 0.0 is as repeatable as the API allows; higher is more varied. Summarising is an *extraction* job — there is a right answer sitting in the ticket — so we keep it low at 0.2. Creative variation in a summary shows up as **invented detail**. The reply stage uses 0.4, because a reply that is word-for-word identical every time reads like a robot. Both judges use **0.0**: a score that changes between runs is useless for tracking quality.

**Trap 3 — the credential is per-node.** There are four OpenAI Chat Model sub-nodes and **each one holds its own credential setting**. Setting it on one does not set the others. We lost time to exactly this: one box went green, the other three still had red warning triangles, and it would have failed halfway through the run.

## Node 6 · Summary Schema (sub-node)

On the connector marked **Output Parser**, click + and add a **Structured Output Parser**. Set Schema Type to **Define using JSON Schema** and paste:

```
{
  "type": "object",
  "properties": {
    "summary": {
      "type": "string",
      "description": "The summary paragraph."
    }
  },
  "required": ["summary"]
}
```

**Why a schema at all?** Without one, the model replies in prose and the next node has to guess where the answer is. The schema forces a predictable shape, so the next node can simply read `summary`. n8n does two things with it: it injects formatting instructions into the prompt, and it validates what comes back.

**Trap 4 — the unclosed code fence.** This is the one that cost us most. The model wrapped its answer in a markdown code block:

```
```json
{"output":{"response":"Dear Customer, ..."}}
```

It opened the block with three backticks and never closed it. n8n strips a code fence only when it finds **both** the opening and closing marks — with only an opening one it leaves the backticks in place, then fails to read the result. The error it reports is *"Model output doesn't fit required*

*format"*, which points at the schema and is entirely misleading; the shape was perfect.

The cure is the last line of every prompt in this workflow: Return raw JSON only. Do not use markdown code fences or backticks. With no fence at all there is nothing to strip.

## Part 3 — The other three stages

---

Build each exactly as above. Only these things differ.

### Node 7 · Evaluation for Summarization

#### Prompt:

```
Original Ticket: "{{ $('Extract Ticket Data').item.json.support_ticket_desc }}"
Ticket Summary: "{{ $json.output.summary }}"
Evaluate the summary.
Return raw JSON only. Do not use markdown code fences or backticks.
```

**Model:** temperature 0.0 , max tokens 500 . **Schema fields:** information\_extraction\_score , information\_extraction\_reasoning , field\_coverage\_score , field\_coverage\_reasoning . Scores are integers 1–3.

**This is "LLM-as-a-Judge".** A second model, given the original *and* the summary, marks the first one's work and must quote the specific detail that justifies each score. Note that it reads the original ticket too — a judge that only sees the summary cannot tell whether anything was invented.

### Node 8 · Response Generation

#### Prompt:

```
Ticket Summary: "{{ $('Summarization').item.json.output.summary }}"
Return raw JSON only. Do not use markdown code fences or backticks.
```

**Model:** temperature 0.4 , max tokens 1500 . **Schema field:** response .

The system message here carries ShopNest's four real support policies verbatim — refunds and returns, delivery delay compensation, wrong or damaged items, and payment failures — plus hard rules: never promise anything the policies do not cover, never invent an order number or date, never engage with a legal threat, never blame the customer or the courier.

**Why reply from the summary, not the ticket?** Two reasons. It is the whole point of summarising first — if the reply still needs the raw ticket, the summary earned nothing. And it keeps abuse, threats and rambling out of the reply path entirely.

**Trap 5 — a format error that was really a length limit.** This stage was originally capped at 600 tokens. A 110–180 word reply, plus the JSON wrapper, plus the parser's own injected instructions, does not fit — so the answer stopped mid-sentence, the JSON was incomplete, and the parser reported "*Model output doesn't fit required format*". Same misleading message as Trap 4, completely different cause.

How to tell them apart: **read the end of the model's output**. Stops mid-sentence → raise the token limit. Complete but wrapped in stray backticks → the fence problem.

## Node 9 · Evaluation for Response Generation

### Prompt:

```
Ticket Summary: "{{ $('Summarization').item.json.output.summary }}"
Generated Response: "{{ $json.output.response }}"
Evaluate the response.
Return raw JSON only. Do not use markdown code fences or backticks.
```

**Model:** temperature 0.0 , max tokens 1200 . **Schema fields:** issue\_addressal\_score , issue\_addressal\_reasoning , resolution\_clarity\_score , resolution\_clarity\_reasoning , overall\_score , overall\_reasoning .

This judge is given **the same four policies** as the writer, so it audits against the rules the writer was actually handed rather than its own idea of them. overall\_score is the sum of the two criteria, so it runs 2-6.



## Part 4 — Assembling the output (3 nodes)

### Node 10 · Extract the Outputs

**Add node → Edit Fields (Set)**, Mode **Manual Mapping**. Add 14 fields. The first four are the ones the brief names:

```
support_ticket_id    {{ $('Extract Ticket Data').item.json.support_ticket_id }}
support_ticket_desc  {{ $('Extract Ticket Data').item.json.support_ticket_desc }}
summarisation        {{ $('Summarization').item.json.output.summary }}
response_generation  {{ $('Response Generation').item.json.output.response }}
```

The other ten are the four criterion scores, the overall score, and each judge's written reasoning — so every score sits on the same row as the text that earned it.

### Node 11 · Convert to File

Operation **Convert to CSV**, Put Output File in Field `data`. This collapses all 30 items into a single file.

### Node 12 · Save the Data

**Read/Write Files from Disk**, Operation **Write File to Disk**, File Path and Name `/data/learner-<your-id>/files/shopnest_ticket_output.csv`, Input Binary Field `data`.

*Note this is a different path from Node 2 — same folder, different filename. Forgetting to change this one is why our first complete run ended in a red box at the very last step.*

## Reading the expressions

Everything in `{{ }}` is a live lookup. Three patterns cover the whole workflow.

| Expression                                                | Means                                                                                   |
|-----------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <code>\$json.output.summary</code>                        | From the node <b>immediately before</b> this one.                                       |
| <code>\$('Summarization').item.json.output.summary</code> | From a <b>named</b> node further back. Use this to reach past the previous node.        |
| <code>.item</code>                                        | "the matching item" — ticket 7's summary lines up with ticket 7's reply, automatically. |

**Why `.output.` is always in the middle.** The Structured Output Parser wraps everything it returns inside a key called `output`. So it is `$json.output.summary`, never `$json.summary`. Miss it and you get blank columns with no error at all — which is exactly what happened to us once during the build.

## Checking it actually worked

The single most useful habit from this whole project: **an execution status is not evidence**. Every failure we hit reported success somewhere. So check the artefact instead.

- **Item counts.** Every stage should say 30 items. A stage showing 1 item has stopped early.
- **Duration.** 120 model calls take minutes. An instant run did nothing.
- **Where the green line stops.** On the canvas, the connectors turn green as data flows. Where they go grey is where execution actually ended.
- **Open the CSV.** Row count, no empty columns, scores inside 1-3, and `overall_score` equal to its two parts added together.

## What your run produced

| Measure                                 | Result                                            |
|-----------------------------------------|---------------------------------------------------|
| Information Extraction / Field Coverage | 2.87 and 2.83 out of 3                            |
| Issue Addressal / Resolution Clarity    | 2.67 and 2.70 out of 3                            |
| Overall                                 | 5.37 out of 6 — 20 of 30 replies scored a clean 6 |
| Invented order numbers                  | zero, across all 30 summaries                     |
| With an order reference vs without      | 5.65 vs 4.80 — the gap is the intake data         |

Built for the ShopNest Global ticket intelligence project. The workflow file, prompts and output checker are in the project repository. Every figure above is measured from your own lab run.